

Robot Data Request – Boise State University

1. Team Registration
 - 1.1. School Name: Boise State University
 - 1.2. Number of Robots: 1
 - 1.3. This form is for robot number 1 of 1
 - 1.4. If more than one robot, describe specific function of this robot: N/A
2. Mobility
 - 2.1. Does your robot have:
 - 2.1.1. Tracks
 - 2.2. Wheels
 - 2.2.1. N/A
 - 2.3. Tracks
 - 2.3.1. Describe anything unusual or unique about your design if it is different than a 2-track system.
 - 2.3.1.1. N/A
 - 2.3.2. Spacing between tracks (mm): 408mm
 - 2.3.3. Length of ground contact for each track (mm): 19.25mm
 - 2.3.4. Width of tread (mm): 170mm
 - 2.3.5. Tread material for tracks: PLA
 - 2.3.6. Do your robot's tracks have grousers: Yes
 - 2.3.7. Tread material for tracks: PLA
 - 2.3.8. Height of grousers (mm): 23.50mm
 - 2.3.9. Spacing between grousers (mm): 18.50mm
 - 2.3.10. Grouser Material: PLA
 - 2.3.11. Which shape best describes your grousers:
 - 2.3.11.1. Straight
 - 2.3.12. Number of driven wheels (per track): 1
 - 2.3.13. Number of idler wheels (per track): 5
 - 2.3.14. How many of the idler or driven wheels are tensioners (per track): 1
 - 2.3.15. Are the tensioners
 - 2.3.15.1. Idlers
 - 2.3.16. Are the tensioner wheels
 - 2.3.16.1. Raised above the surface
 - 2.3.17. Are the driven wheels
 - 2.3.17.1. Raised above the surface
 - 2.3.18. How many idlers are raised above the surface: 5
 - 2.3.19. How many idlers are in ground contact: 0
 - 2.3.20. Diameter of the driven wheels (mm): 98mm
 - 2.3.21. Diameter of the idler wheels (mm): 70mm
 - 2.3.22. Motor torque for the track's driven wheels: 15NM

- 2.3.23. Gear ratio from motor to driven wheels: 24:1
 - 2.3.24. Type of gearing to driven wheels: Planetary
- 2.4. Traction Control
 - 2.4.1. Please identify and describe any implemented methods or systems designed to maintain tread or wheel traction
 - 2.4.1.1. None
- 3. Steering
 - 3.1. If your robot has tracks, then tank steering (i.e., skid steering) is normal, but explain if the robot uses tracks with a unique steering method instead
 - 3.1.1. N/A
 - 3.2. If wheels, check which applies:
 - 3.2.1. N/A
- 4. Suspension
 - 4.1. Is the robot's suspension:
 - 4.1.1. None
 - 4.2. Describe suspension (lead springs, rocker bogey, MacPherson strut, etc., or describe)
 - 4.2.1. N/A
- 5. Chassis
 - 5.1. Materials used: Aluminum extrusions & aluminum paneling with an acrylic top
 - 5.2. Construction: Body on Frame
 - 5.3. Welded bolted, riveted, etc.: Bolted
 - 5.4. Majority ground clearance in cm: 21.1cm
 - 5.5. Minimum Ground Clearance in cm: 5.3cm
- 6. Autonomy
 - 6.1. Which autonomous operations will be attempted:
 - 6.1.1. Full autonomy - Entire Run
 - 6.2. Development Environment
 - 6.2.1. Was a basic prototype built to start autonomy development early? When was it ready for autonomy development?

No prototype was constructed to start autonomy development early.
 - 6.2.2. Was a simulation model used for autonomy development?
 - 6.2.2.1. What simulator (ex. Gazebo, CARLA)? When was the simulation model ready for autonomy development?

The simulator Gazebo was used for autonomy development. A simulation model was used, and was ready for autonomy testing on March 1st.
 - 6.2.3. When was the final robot ready for autonomy development?

The final robot will be ready for autonomy development on April 17th.

- 6.3. Autonomy Sensors -Describe the sensors you are using in your autonomy approach. For Example Perception - Lidar - scanning lidar, flash lidar; depth/stereo camera - Kinect; mono (RGB) camera; sonar. Localization – Ranging radios – ex. Ultra High Band; IMU, powertrain odometry.

6.3.1.

Sensor Type	Sensor Description	Sensor Use
Lidar	Solid State 3d lidar, Solid State 2d lidar, 360° 3d lidar.	Obstacle and crater detection.
Stereo Camera	Intel Realsense D435	Obstacle and crater detection.
IMU	Vectornav VN-200	Movement sensing, pose updating.
Beacon	Vive 2.0 + Valve Beacon	Positioning in the environment.

- 6.4. Compute - Describe the hardware and software you are using to run your autonomy and automation system. Example: Quad core Intel i7 processor running Ubuntu 20.04 operating system. Software is a mixture of Python and C++. If you have multiple computers or computing architectures, describe below. Please include your lower level I/O and machine control as well as the autonomy level.

The hardware system is composed of an NVIDIA Jetson Orin Nano, Two Raspberry PI 4's, and a Beelink Mini PC. All systems are running Ubuntu 22.04. The Beelink is using an Intel x86_64 architecture, while the other computers use ARM64.

6.4.1.

Lower Level I/O and Machine Control
All sensor input will be received by computers through USB port. Computers will be connected to a LAN by an Ethernet switch located on the robot, one of the Raspberry Pi will function as a router to the wifi access point located externally.
Autonomy Layer
All interprocess communication between the computers on the robot will be handled through the ROS2 Iron framework. This is the framework that all autonomy software is written for.

- 6.5. Architectural Approach - Goal is to capture and understand the general approach to autonomy and automation in your architecture. Looking for how you use/apply the sensor modalities described in the sensor section above. Also, sensor/data fusion approaches. What open-source software modules were leveraged? What compute platform from above did you leverage for each area?

6.5.1. Autonomy

- 6.5.1.1. Perception - Description around your approach to perception - object detection, terrain mapping, etc. Examples - Terrain mapping (ground plane estimation based on RANSAC with identification of positive or negative returns above or below the ground plane), Object detection (classical approaches ex. camera and edge detections, or AI/ML approach). What sensor modality (sensor type) is used in the approach? Are you using multiple modalities and fusing? What method for fusing?

Lidar and camera sensors will collect point cloud observations of the environment. These raw observations will then pass through a filter that removes any data outside of the bounds of the competition area, to conform with the requirements of sections 6.3-6.4 of the autonomy section of the Lunabotics 2025 Guidelines. Filtered observations will then be passed to a central computer that transforms them into the map's reference frame and places them in an occupancy grid map. Lidar data will be assumed to only contain obstacles and be placed 'as is', while point cloud data will only be added as obstacles if it lies above or below a certain z-axis range chosen to filter the ground plane but still detect holes and smaller rocks. This occupancy grid will contain all data needed for remaining autonomy and localization. The occupancy grid will additionally be updated periodically via a GraphSLAM algorithm that will rearrange previous sensor readings based on improved positional estimates. The Ceres nonlinear optimization library was leveraged for the update step of the GraphSLAM system.

- 6.5.1.2. Localization - Description of your approach to determine where you are within the arena. Examples - Fiducials located at the collector sieve to give location with fusion with IMU for orientation. Describe fiducials and how they are used. What sensor modalities are used? How are you fusing sensor inputs -

Kalman Filter, Extended Kalman Filter, Particle Filter, other...? Do you leverage any visual (lidar/camera) odometry?

Localization will be done using the Vive 2.0 tracker via the Valve Beacon while the robot is able to collect accurate measurements of it. When this information is unavailable, the system will switch to using a Kalman Filter that fuses the previous Vive odometry inputs and the Vectornav IMU inputs to provide continued position data. GraphSLAM and scan matching via the ICP algorithm will also be used to correct for drift in the Kalman filter. The robot_localization software package in the ROS2 ecosystem is used as the Kalman filter.

- 6.5.1.3. Navigation Planning - How do you plan and execute your navigational paths? How do you determine the path - path planning? How do you execute the path-path tracking? How do you leverage the perception data (object detection, etc) into this decision-making?

Path planning is done in two phases. First, an A* search is done over the current occupancy grid to determine a path for the robot to take through the environment. This path is then passed to the smoothing algorithm, which is an implementation of the original Elastic Band algorithm. Smoothing is done over the same occupancy grid. The final smooth path is then used by the path follower to arrive at the goal. The path follower currently uses a simple method of turning to face the next waypoint, and then driving towards it. This method may change depending on time constraints.

- 6.5.1.4. Excavation planning - How do you determine where to dig? Where to go next if you run several cycles per competition run?

The dig zone for the competition is initially partitioned into several 'dig areas'. The robot will then perform an initial passover of the environment to identify any potential obstacles present in the dig/dump zones. The dig areas will be updated and stored in a queue in an order that ensures digging holes in one dig area will not impact the reachability of another. The robot proceeds by

digging in the first area in the queue, moving to the dump area to dump, and then repeating this process until it runs out of dig areas.

6.5.2. Automation

- 6.5.2.1. Excavation - Lower-level planner and controls around performing a single excavation cycle. Once you've determined where excavation should occur, how do you execute a single cycle? What onboard parameters do you leverage - actuator force or velocity, etc? How do you make decisions for the excavation cycle from these parameters? Depth of cut, rate of cut, etc

A single cycle will be determined by the bucket drum being filled up to a predetermined weight of regolith, implemented as a set value. This value will ideally be 9kg. The weight of regolith in the drum will be determined by 4 strain gauges, 2 placed on each arm holding up the bucket drum, and calibrated with known weights to measure the regolith fill capacity correctly. To begin a cycle, the drum will be lowered by its arms until the strain gauges and stepper motor drivers measure a load on the drum from touching the ground. The arms will then be lowered such that the bottom of the drum is lowered 40-90% of its total cut depth, in this case 22.6mm to 50.85mm. Once the drum is lowered, the robot will be controlled to move forward linearly at an optimal set speed of 13cm/s, and the drum to spin at an optimal collecting speed of 55rpm. These speeds are chosen to keep an optimal tangential velocity to linear velocity ratio for the drum of 8.5. This ratio will be enforced on any non-nominal linear or rotational speeds as well. If the drum is unable to collect regolith at the desired rate for a given cut depth, the cut depth will be increased accordingly. Similarly if the drum motors are exceeding current specifications, the cut depth or cut speeds will be decreased accordingly to offset this. Once the strain gauges measure the drum to have a sufficient amount of regolith collected, the drum will then be raised from the ground and be stopped from spinning.

- 6.5.2.2. Dump - Lower-level planner and controls around performing a single dump cycle. How do you align to the berm location? What onboard parameters do you leverage - actuator position, velocity, etc? How do you make decisions for the dump cycle from these parameters?

Rigidly fixed bucket drum on a known radius that the robot has positional knowledge of. We can currently control the arms and the bucket drum's position, velocity, and acceleration. From this, the robot can use the known area map and preconfigured dump zone coordinates to align the bucket drum with the dump location.

7. Regolith Mining System
 - 7.1. Does your robot have:
 - 7.1.1. Bucket Drum
 - 7.2. If other, please describe. Additionally, please describe any other strategies or unique features of your robot.
 - 7.2.1. Our bucket drum includes soft internal manifolds to allow for a greater carry capacity and decrease the amount of spill from the system.
8. Regolith Transport System
 - 8.1. Will your robot:
 - 8.1.1. Transport regolith in the mining device itself
 - 8.2. If "other" was checked please describe
 - 8.2.1. N/A
 - 8.3. Estimate maximum amount of regolith that can be transported in one mining cycle (kg)
 - 8.3.1. 9kg
9. Regolith Unloading System
 - 9.1. Does your robot have
 - 9.1.1. Reverse rotate bucket drum
 - 9.2. If "other" was checked please describe
 - 9.2.1. N/A
 - 9.3. Does it use methods to enhance the discharge flow?
 - 9.3.1. No
 - 9.4. Methods used to enhance the discharge flow:
 - 9.4.1. None
 - 9.5. If special coatings or "other" was checked, please describe
 - 9.5.1. N/A
10. Dust Mitigation Features
 - 10.1. Describe any dust mitigation features designed into the robot
 - 10.1.1. Seals
11. Other Unique Features
 - 11.1. Please describe any other unique features that may be helpful to NASA in learning how systems and materials work for space mining.

12. Energy Measurement

12.1. Provide the make and model of your energy

12.1.1. BrtPower 40V 30Ah LiFePO4 ([Link](#))